Golf

Agentic AI Security & Governance

Enterprise guide
to secure agentic AI

2026

Two Realities in Enterprise AI

Every enterprise we talk to is in one of two situations.

REALITY A

You've blocked AI tools.

Security said no. Too much risk, no visibility, no way to control what connects to what. The answer was to block Claude, ChatGPT, Cursor, Copilot - all of it.

The problem: your employees are using these tools anyway. On personal devices, personal accounts, outside your network. You don't see it, you can't audit it, and you have zero control over what data flows through. The block didn't reduce risk - it just eliminated your visibility and governance controls.

Meanwhile, your competitors are building better and quicker. 65% of enterprises now use generative AI regularly - double from last year (McKinsey 2025).

REALITY B

You've approved AI tools.

You said yes to some combination of Claude, ChatGPT, Cursor, Copilot, Warp, Claude Code. Maybe through official procurement, maybe through a team-level decision, maybe it was just a one-off decision.

These tools are now connecting to your systems - GitHub, Slack, Jira, internal APIs. Each agent has its own integration settings. Each one is a separate authentication flow, separate permissions model, separate audit trail (if there's an audit trail at all).

Your security team cannot answer basic questions: Which AI tools are connected to GitHub? Who approved those connections? What data flowed through them last month?

You enabled AI. You're also flying blind.

Both paths lead to the same place.

Whether you blocked everything or approved everything, you need a strategy for governing how AI tools connect to your systems. That's what this framework is for.

How agents connect to your systems

AI tools get better with context. The more relevant context you give them - your codebase, your docs, your Slack history - the better the output. Integrations are how agents get that context automatically, without your employees copy-pasting everything.

To get real value from AI tools, they need integrations with your relevant systems. And in 2026, there's a standard protocol for how those integrations work: MCP.

What is MCP?

Model Context Protocol. Think of it as USB-C for AI agents - one universal standard instead of custom integrations for every tool.

Before MCP, if you wanted 10 AI apps to connect to 50 tools, you needed 500 custom integrations. With MCP, you need 60: each app implements MCP once, each tool implements MCP once, and they all work together.

MCP is now an open standard under the Linux Foundation's Agentic AI Foundation.

Members include Anthropic, OpenAI, Google, Microsoft, AWS, and Cloudflare. Every major AI agent - Claude, ChatGPT, Cursor, Copilot, Warp, Claude Code - has adopted it.

This isn't a bet on emerging technology.
It's the standard.

Two dimensions to understand

DEPLOYMENT TYPE

Remote Hosted as a service. Centralized security, audit logging, access controls, always-on. This is what enterprises should use.

Local Runs on the developer's laptop. No central control, no audit trail, dies when the laptop closes. Fine for experimentation. Not for production.

OWNERSHIP

First-party You build and host it. Full control over code, auth, infrastructure. Connects to your internal systems.

Third-party Vendor builds and hosts it. You integrate via URL. Less control, but faster to deploy. Examples: GitHub's official MCP server, Slack's MCP server.

These two dimensions give you four quadrants. Knowing where each integration sits determines your risk profile and governance requirements.

	REMOTE	LOCAL
First-party (You build it)	Internal APIs, databases	Dev tooling
Third-party (External)	GitHub, Linear, Slack	Random GitHub repos

Remote and local servers have different risk profiles. Remote servers allow centralized security, audit logging, and access controls. Local servers lack these by default, but employees will still use them. Your governance and enforcement approach needs to account for both.

The Fragmentation Problem

MCP solved the integration problem. It didn't solve the governance problem.

You might look at the previous section and think: "Great, one universal standard. All my agents speak the same protocol. Problem solved."

Not quite.

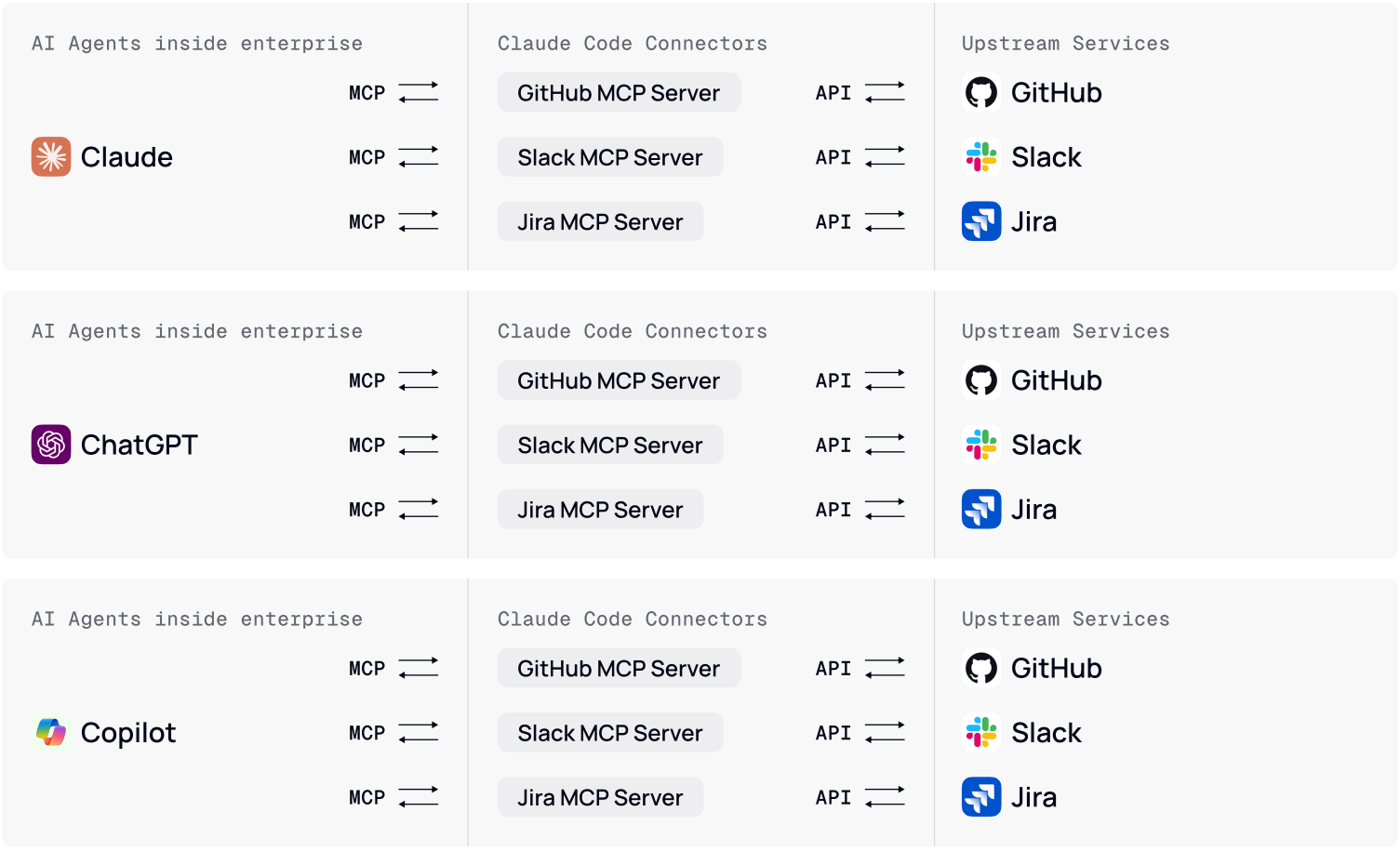
MCP standardized how agents connect to tools. It didn't standardize where those connections are controlled. Each agent is still a separate MCP client with its own configuration.

Cursor stores MCP connections in a local config file. Claude manages them in a connectors panel. Warp has their own settings. ChatGPT handles integrations differently. Each agent maintains its own list of which servers it connects to, which credentials it uses, which permissions it has.

So if you've approved five agents across your organization, you now have five separate control points. Five places where someone can add a new integration. Five different ways authentication is handled. Five different places to check if you want to audit what's connected to what.

MCP gave you interoperability. It didn't give you centralized governance.

The protocol is standardized. The control plane is fragmented. Close the gap to manage agent integrations across teams with a single view.



What We're Seeing Go Wrong

These aren't theoretical risks. They're patterns we see repeatedly across enterprises adopting AI agents.

I.

Shadow AI

Without clear policies, employees connect whatever they find. Someone discovers an MCP server that connects their agent to Salesforce. They set it up. It works. They tell their team.

Now six people are using an integration that security has never reviewed, IT doesn't know exists, and nobody is monitoring.

This is shadow IT, but accelerated and scaled. An employee can connect an AI agent to sensitive systems in five minutes. No ticket, no approval, no guaranteed audit trail.

II.

Unvetted Servers

An engineer wants to connect their coding agent to a tool. They Google "MCP server for [tool name]." They find a GitHub repo. It looks legitimate - maybe it has a few hundred stars.

They download it and run it locally.

No security review. No code audit. No verification that this is an official server from the vendor. And because it's running locally, there's no authentication layer, no access controls, no audit logs.

Whoever runs it gets full access to whatever it connects to.

In 2025 alone, multiple MCP servers on GitHub were found to contain command injection vulnerabilities - CVE-2025-53107, CVE-2025-5277, CVE-2025-53818, CVE-2025-69256.

These weren't obscure projects. They were servers that developers were actively using.

III.

Rug Pulls

You vet a third-party MCP server today. It looks fine. You approve it.

Next week, the maintainer pushes an update. New tools added. Behavior changed. Maybe intentionally malicious, maybe just careless. Your agents are now running different code than what you reviewed.

This is called a 'rug pull' - a supply chain attack where a trusted tool is later updated with malicious behavior (Kaspersky, 2025).

There's no versioning enforcement. No change detection. No alert when a server you depend on modifies its capabilities. The MCP server you approved last month isn't necessarily the MCP server running today.

IV.

PII Leakage to Model Providers

Every agent request flows through a model provider - Anthropic, OpenAI, Google. When an employee asks their agent to "summarize this customer file," the contents of that file are sent to the model provider.

If that file contains social security numbers, credit card data, health records, or other PII - it's now sitting on a third-party server. In the worst case, it could be used to train third-party large language models.

For fintechs, healthcare companies, and any regulated industry, this can be a compliance violation.

Without a governance layer, there's no inspection of what's flowing out. No PII detection. No scrubbing of sensitive data before it reaches the model provider. No DLP policies applied.

V.

No Audit Trail

When an incident happens, you need to answer questions: What data was accessed? Which systems were queried? Who initiated the request? What was returned?

With fragmented agent integrations, you can't answer any of this. There are no logs. There's no central record of what data flowed through which agent to which system.

This isn't just a security problem. It's a compliance problem. When auditors ask how you govern AI access to sensitive systems, "we don't track that" is not an acceptable answer.

III.

Prompt Injection via Integrations

In May 2025, security researchers demonstrated that the official GitHub MCP server - with over 14,000 stars - was vulnerable to prompt injection attacks.

An attacker could create a malicious issue in a public repository. When a developer asked their agent to "check open issues," the agent would read the malicious content, get hijacked, and leak data from private repositories.

Security researcher Simon Willison called it a "lethal trifecta for prompt injection - access to private data, exposure to malicious instructions, and the ability to exfiltrate information."

The attack didn't require compromising the MCP server itself. It exploited the fact that agents read untrusted content and have access to sensitive systems.

Without monitoring at the integration layer, you'd never know it happened.

The Principle

These problems have a common root: fragmented control and limited visibility across multiple agents.

The solution is a single governance layer that all agents route through.



All agents route through one gateway. Centralized auth, logging, and policy enforcement.

One control point for authentication, permissions, data inspection, and audit logging - regardless of which agent an employee uses.

The next section details what that looks like in practice.

What Governance Looks Like

This section outlines the components of an effective governance framework for AI agent integrations. Think of it as a checklist - what you need in place to enable agents safely.

Centralized Integration Control

All agent connections route through a single gateway. Doesn't matter if an employee uses Claude, Cursor, ChatGPT, or Warp - when they connect to GitHub, Slack, or an internal system, the connection goes through the same control point.

One inventory. One place to see which agents access which systems. One place to make changes.

Classification & Approval

Not all integrations carry the same risk. Classify by ownership (first vs third-party), deployment model (remote vs local), risk level (read-only vs write/delete), and vendor/registry reputation.

Route approval workflows through your existing systems - ServiceNow, Jira, whatever your teams already use.

The goal is to extend your organization's existing governance to AI tools and agents while minimizing friction, bureaucracy, and complexity.

Authentication & Access Control

Integrate with your existing identity provider. Employees authenticate via SSO. Enforce role-based access at the gateway - a marketing intern shouldn't have the same permissions as a staff engineer.

Least privilege by default.

Data Protection

Inspect data flowing through agent integrations. Detect and mask PII, credentials, and financial data before it reaches model providers.

Extend your existing compliance requirements - SOC2, HIPAA, PCI, GDPR - to the agent integration layer.

Audit & Visibility

Log every request: who, which agent, integration, data, and when.

This gives you incident response capability and the audit trail compliance teams need.

Enforcement (Phased Approach)

You don't have to implement everything on day one.

PHASE 1

Policy

Document the policy. Communicate that all agent integrations should go through the governance layer. Update security training.

PHASE 2

Endpoint Configuration

Use MDM (Jamf, Intune) to deploy pre-configured agent setups that point to the gateway by default. Make compliance the path of least resistance.

PHASE 3

Network Enforcement

Block direct outbound connections to MCP server endpoints at the firewall/proxy level. Whitelist only gateway IP addresses.

Start where you can, build toward full enforcement.

Self-Assessment

Before evaluating solutions, understand your current state. These questions help identify gaps.

Visibility	Can you list every AI agent employees are using today? Do you know which internal & external systems those agents connect to? Can you see what data flows through those connections?
Control	Is there an approval process for new agent integrations? Can you revoke access to a specific integration across all agents at once? Do you control which tools within an integration are available to which roles?
Compliance	Could you produce an audit log of all agent activity for the past 90 days? Are you confident PII isn't flowing to model providers through agent integrations? Do your current DLP policies cover agent connections?
Incident Response	If an employee's credentials were compromised, could you identify what an attacker accessed through their agent integrations? Can you answer "what did this agent do between 2pm & 4pm last Tuesday"?
Enforcement	Are agent integration policies documented? Are those policies enforced technically, or just on paper? Could an employee bypass governance by configuring their own connections?
The uncomfortable ones	If an auditor asked how you govern AI agent access to production systems, what would you show them? Could a departing employee exfiltrate data through an agent integration on their last day? Do you know the difference between what agents can access vs what they actually access?

If most of these are "no" or "I don't know"
- you have a governance gap.

The question is whether you close it proactively or after an incident.

Golf builds the governance layer for AI agent integrations.

LET'S TALK ABOUT YOUR AGENTIC GOVERNANCE

golf.dev LinkedIn wojciech@golf.dev

Golf